

Mapeamento Completo de Elementos (Pratice)

Site: [Demoblaze](#) 

Requisição: Cada elemento abaixo utiliza uma estratégia distinta de mapeamento, com base no HTML real da aplicação. A justificativa descreve por que cada estratégia foi utilizada.

1. ID único

- **Nome do elemento:** Modal de Login
 - **ELEMENT:** `<div class="modal fade" id="loginModal">`
 - **SELECTOR:** `#loginModal`
 - **XPAT:** `//*[@id='loginModal']`
 - **Estratégia: ID**
 - **Justificativa:**
A estratégia de ID é usada porque o elemento possui um identificador único e semântico, garantindo alta precisão e estabilidade na localização.
-

2. Seletor customizado (atributo alternativo: `onclick`)

- **Nome do elemento:** Botão “Log in” (ação)
 - **ELEMENT:** `<button onclick="login()" class="btn btn-primary">Log in</button>`
 - **SELECTOR:** `button[onclick='login()']`
 - **XPAT:** `//button[@onclick='login()']`
 - **Estratégia: Atributo customizado**
 - **Justificativa:**
Utilizamos um atributo específico (`onclick`) como substituto de um `data-*`, pois ele identifica de forma consistente a ação do botão.
-

3. Atributo `id` no lugar de `name`

- **Nome do elemento:** Campo de username
 - **ELEMENT:** `<input id="loginusername">`
 - **SELECTOR:** `#loginusername`
 - **XPAT:** `//*[@id='loginusername']`
 - **Estratégia: ID**
 - **Justificativa:**
O atributo `name` não está presente, e o `id` é único e confiável. Por isso, usamos `id` como substituto seguro.
-

4. Classe (`class`)

- **Nome do elemento:** Link “Log in” no menu

- **ELEMENT:** `<a class="nav-link" id="login2">Log in</a>`
- **SELECTOR:** `.nav-link#login2`
- **XPATh:** `//a[@id='login2']`

- **Estratégia: Class name**

- **Justificativa:**

A classe `.nav-link` é padrão no layout, e combinada com o ID `login2`, forma um seletor robusto e preciso.

5. CSS Selector Avançado (Descendant) [🔗](#)

- **Nome do elemento:** Categoria “Phones”

- **ELEMENT:** `<a class="list-group-item">Phones</a>`

- **SELECTOR:** `.list-group > a`

- **XPATh:** `//div[@class='list-group']/a[contains(text(), 'Phones')]`

- **Estratégia: Descendant Selector**

- **Justificativa:**

Reflete corretamente a hierarquia entre o container `.list-group` e seus filhos `<a>`. Facilita manutenção e leitura.

6. Operadores de atributo (`^=` , `*=`) [🔗](#)

- **Nome do elemento:** Campo de login

- **ELEMENT:** `<input id="loginusername">`

- **SELECTOR:** `[id^="login"]`

- **XPATh:** `//*[starts-with(@id, 'login')]`

- **Estratégia: Operadores de atributo**

- **Justificativa:**

Ideal quando os atributos seguem padrões previsíveis. Permite encontrar todos os campos com prefixos como `login`.

7. Pseudo-classe `:not` [🔗](#)

- **Nome do elemento:** Campos de input visíveis

- **ELEMENT:** `<input type="text"> / <input type="password">`

- **SELECTOR:** `input:not([type='hidden'])`

- **XPATh:** `//input[not(@type='hidden')]`

- **Estratégia: CSS** `:not`

- **Justificativa:**

Permite filtrar inputs técnicos ou invisíveis, garantindo que apenas campos interativos sejam usados nos testes.

8. XPath com texto [🔗](#)

- **Nome do elemento:** Botão “Log in” (ação final)

- **ELEMENT:** `<button>Log in</button>`

- **XPATh:** `//button[text()='Log in']`

- **Estratégia: XPath com texto**

- **Justificativa:**

Quando atributos são genéricos ou ausentes, o conteúdo textual se torna uma boa referência — desde que fixo.